

Protocol Audit Initial Report

Version 1.0

CyberGhost

October 17, 2026

Protocol Audit Report

CyberGhost

October 13, 2026

Prepared by: CyberGhost

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] Incorrect fee calculation in the `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fee.
    * [H-2] Lack slippage protection in `TSwapPool::swapExactOutput`, causes users to potentially receive way fewer tokens.
    * [H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens, causing users to receive the incorrect amount of tokens.
    * [H-4] In `TSwapPool::_swap` the extra tokens given to users after every `SWAP_COUNT_MAX` breakes the protocol invariant of `x * y = k`.

- Medium
  - [M-1] `TSwapPool::deposit` function is missing deadline check, causing transactions to complete even after the deadline.
- Low
  - [L-1] `TSwapPool::LiquidityAdded` event has parameters out of order.
  - [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given.
- Informational
  - [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is used nowhere and should be removed.
  - [I-2] Lacking zero address check.
  - [I-3] `PoolFactory::createPool` should use `.symbol()` for `liquidityTokenSymbol` instead of `.name()`.
  - [I-4] `PoolFactory::PoolCreated(address,address)` has address parameters but no `indexed` parameters.
  - [I-5] `TSwapPool::TSwapPool__WethDepositAmountTooLow` accepts `minimumWethDeposit` which is a constant variable and therefore not required to be emitted.
  - [I-6] The `poolTokenReserves` local variable in `TSwapPool::deposit` is used nowhere and should be removed.
  - [I-7] `TSwapPool::deposit` function updates the `liquidityTokensToMint` variable to return after making a potential external call, violating the CEI Pattern.
  - [I-8] `TSwapPool::getOutputAmountBasedOnInput`, `TSwapPool::getInputAmountBasedOnOutput` and `TSwapPool::swapExactInput` are missing a documentation (NatSpec) while they are the core functions of the entire protocol.
  - [I-9] The variables to calculate fees within the `TSwapPool.sol` are not declared as constants.
  - [I-10] `TSwapPool::swapExactOutput` function should be declaread as an `external` function, due to the fact that it is used nowhere within the contract.
  - [I-11] The `TSwapPool::swapExactOutput` NatSpec is missing the `deadline` parameter, causing the NatSpec to be incomplete.
  - [I-12]: Large Numeric Literal

Protocol Summary

The protocol acts as a decentralized exchange based on the `UniswapV2` which utilizes the constant product market maker format (`CPMM`) and has the key invariant of `x * y = k` where the multiplication of the reserves of two tokens must always be the same after and before each transaction (the same k, hence 'constant'), preserving the ratio of a trading pair. Liquidity providers are able to deposit money to TSwap Pools where each smart contract (pool) is an exchange between WETH and any ERC20 token. Pools can be created by the `PoolFactory`. In order to create a pool, users must specify an ERC20 token address.

Disclaimer

This security review is a point-in-time assessment conducted independently on a time-boxed basis. The findings, architectural notes, and Proofs of Concept contained herein reflect the state of the codebase at the exact time of analysis and do not guarantee an absolute absence of vulnerabilities.

This document is provided for informational and security-hardening purposes only. It does not constitute financial, legal, or investment advice, nor does it represent an endorsement of the underlying protocol, team, or token economics.

The independent security researcher disclaims all liability for any direct or indirect financial loss, exploited vulnerabilities, or protocol failures arising from the deployment, modification, or interaction with the reviewed smart contracts. Security is an ongoing process, not a final destination.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
Commit Hash: e643a8d4c2c802490976b538dd009b351b1c8dda
```

Scope

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

Roles

- Liquidity Providers: Users who have liquidity deposited into the pools. Their shares are represented by the LP ERC20 tokens. They gain a 0.3% fee every time a swap is made.
- Users: Users who want to swap tokens.

Executive Summary

During the review of the codebase, a severe disruption of functionality has been found. The protocol team must follow up the security report and add all the respective recomendations before moving forward.

P.S: Awesome Work, Patrick!

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2
Info	12
Gas	0

Severity	Number of issues found
Total	19

Findings

High

[H-1] Incorrect fee calculation in the `TSwapPool::getInputAmountBasedOnOutput` causes protocol to take too many tokens from users, resulting in lost fee.

Description: The `TSwapPool::getInputAmountBasedOnOutput` function is intended to calculate the amount of tokens a user should deposit given an amount of output tokens. However, the function currently miscalculates the resulting amount. When calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: Protocol takes more fees than expected from users.

Proof of Concept: The test below demonstrates the exact flaw in the `TSwapPool::getInputAmountBasedOnOut`
:

PoC

1. A liquidity provider deposits 100 WETH and 100 poolTokens;
2. A user wants to swap their pool tokens for 5 WETH;
3. A user calls the `TSwapPool::swapExactOutput` function which in turn calls `TSwapPool::getInputAmountBasedOnOutput`;
4. The expected input amount a user must put into for their 10 WETH: ~ 5.27 poolTokens;
5. The actual input amount the protocol requires a user to pay: ~ 52.78. **This is 10 times more than intended to be.**

Add the following to the `TSwapPool.t.sol`:

```
function testIncorrectFeeCalculation() public {
      vm.startPrank(liquidityProvider);
      weth.approve(address(pool), 100e18);
      poolToken.approve(address(pool), 100e18);
      pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
      vm.stopPrank();

      // A user wants to swap their pool tokens for 5 WETH.
```

```
        // The correct amount of pool tokens a user should put into the protocol based on x*y=k:
        // delta x = ((inputReserves * outputAmount) * 1000) / ((outputReserves - outputAmount) * 997)
        // delta x = ((100 * 5)) * 1000) / ((100 - 5) * 997)
        // delta x = 500,000 / 94,715 =  ~ 5.27

        // Expected: ~ 5.27
        // Required: 52,789948793749670062
        // The Difference is 50 Tokens!!!
        uint256 actualInput = pool.getInputAmountBasedOnOutput(
            5e18,
            100e18,
            100e18
        );
        console2.log("Actual input: ", actualInput);

        // Let's simulate a user executes the swapExactOutput function
        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        vm.expectRevert();
        pool.swapExactOutput(poolToken, weth, 5e18, uint64(block.timestamp));
        vm.stopPrank();
        // And of course the tx reverts, while in the normal conditions, the user has even more than needed.
}
```

Recommended Mitigation:

```
function getInputAmountBasedOnOutput(
    uint256 outputAmount,
    uint256 inputReserves,
    uint256 outputReserves
)
    public
    pure
    revertIfZero(outputAmount)
    revertIfZero(outputReserves)
    returns (uint256 inputAmount)
{
    return
-       ((inputReserves * outputAmount) * 10_000) /
+       ((inputReserves * outputAmount) * 1_000) /
        ((outputReserves - outputAmount) * 997);
}
```

[H-2] Lack slippage protection in `TSwapPool::swapExactOutput`, causes users to potentially receive way fewer tokens.

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`, where the function specifies the `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If market conditions change before a transaction processes, the user could get a much worse swap.

Proof of Concept:

1. The price of 1 WETH right now is 1,000 USDC;
2. User inputs `swapExactOutput` loking for 1 WETH
    1. inputToken = USDC
    2. outputToken = WETH
    3. outputAmount = 1e18
    4. deadline = whatever
3. The function does not offer a `maxInputAmount`;
4. As the transaction is pending in the mempool, the market changes! And the price moves HUGE -> 1 WETH = 10,000 USDC. 10x more than the user expected.
5. The transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC.

Recommended Mitigation: We should include a `maxInputAmount` so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
    function swapExactOutput(
        IERC20 inputToken,
        IERC20 outputToken,
        uint256 outputAmount,
+       uint256 maxInputAmount,
        uint64 deadline
    )
        public
        revertIfZero(outputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 inputAmount)
    {

    inputAmount = getInputAmountBasedOnOutput(
        outputAmount,
        inputReserves,
        outputReserves
```

```
18          );
19      +   if (inputAmount > maxInputAmount) {
20      +       revert();
21      +   }
22
23          _swap(inputToken, inputAmount, outputToken, outputAmount);
24
25          }
```

[H-3] `TSwapPool::sellPoolTokens` mismatches input and output tokens, causing users to receive the incorrect amount of tokens.

Description: The `sellPoolTokens` function is intended to allow users easily sell their pool tokens and receive WETH in exchange. Users indicate how many pool tokens they're willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount.

This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

PoC

1. Let's say the user wants to swap 4 poolTokens -> WETH; Currently there are 100 WETH and 100 poolTokens in the protocol.
2. Whatever the current ratio is, the function will calculate how many poolTokens the user has to put in to receive 4 WETH. So instead of calculating how many tokens send to the user based on that 4 tokens which they specified as an argument, the function will calculate how many poolTokens the user will be charged to receive 4 WETH.
3. Parameters: inputToken: poolToken outputToken: weth outputAmount: 4 tokens deadline: whatever
4. Let's calculate how many pool tokens will be charged to the user: delta x = (100 * 4) * 1000 / (100 - 4) * 997 delta x = 400,000 / 95,712 delta x = ~ 4.18 poolTokens

A total of 4.18 pool tokens will be charged to the user who intends to sell 4 pool tokens. This is not the intended behavior!

Place the following into the `TSwapPool.t.sol`:

```
function testIncorrectSwapImplementationInSellPoolTokens() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 userWethStartingBalance = weth.balanceOf(user);
    uint256 userPoolTokensStartingBalance = poolToken.balanceOf(user);
    console2.log("Starting WETH: ", userWethStartingBalance);
    console2.log("Starting PT: ", userPoolTokensStartingBalance);

    uint256 amountOfPoolTokensToSwap = 4e18;

    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    pool.sellPoolTokens(amountOfPoolTokensToSwap);
    vm.stopPrank();

    uint256 userWethEndingBalance = weth.balanceOf(user);
    uint256 userPoolTokensEndingBalance = poolToken.balanceOf(user);
    console2.log("Ending WETH: ", userWethEndingBalance);
    console2.log("Ending PT: ", userPoolTokensEndingBalance);

    uint256 poolTokensTaken = userPoolTokensStartingBalance -
        userPoolTokensEndingBalance;
    console2.log("Amount of PT taken: ", poolTokensTaken);

    assertEq(userWethEndingBalance, userWethStartingBalance + 4e18);
    assert(
        userPoolTokensEndingBalance !=
            userPoolTokensStartingBalance - amountOfPoolTokensToSwap
    );
}
```

Test Result

```
[PASS] testIncorrectSwapImplementationInSellPoolTokens() (gas: 462305)
Logs:
  Starting WETH:  1000000000000000000
  Starting PT:  1000000000000000000
  Ending WETH:  1400000000000000000
  Ending PT:  5820795720494817787
  Amount of PT taken:  4179204279505182213 -> This is not the amount
     that being set for the exchange.
```

Recommended Mitigation: Consider changing the implementation to use the `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`).

```
function sellPoolTokens(
    uint256 poolTokenAmount,
+   uint256 minWethToReceive
) external returns (uint256 wethAmount) {
    return
+       swapExactInput(
+           i_poolToken,
+           poolTokenAmount,
+           i_wethToken,
+           minWethToReceive,
+           uint64(block.timestamp)
+       )
}
```

Additionally, it might be wise to add a `deadline` parameter to the function as there is currently no `deadline`.

[H-4] In `TSwapPool::_swap` the extra tokens given to users after every `SWAP_COUNT_MAX` breakes the protocol invariant of `x * y = k`.

Description: The protocol follows a strict invariant of `x * y = k`. Where: - `x`: The balance of the pool token; - `y`: The balance of WETH; - `k`: The constant product of the two balances.

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the `k`. However, it is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained.

The following block of code is responsible for the issue:

```
    swap_count++;
    if (swap_count >= SWAP_COUNT_MAX) {
        swap_count = 0;
        outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000)
            ;
    }
```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps collecting the extra incentive tokens given out by the protocol.

More simply put, the protocol's invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of `1_000_000_000_000_000_000` ; 2. That user continues to swap untill all the protocol funds are drained.

Proof Of Code

Place the following into the `TSwapPool.t.sol`:

```
function testInvariantBroken() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputWeth = 1e17;

    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
```

```
51          outputWeth,
52          uint64(block.timestamp)
53      );
54      pool.swapExactOutput(
55          poolToken,
56          weth,
57          outputWeth,
58          uint64(block.timestamp)
59      );
60      pool.swapExactOutput(
61          poolToken,
62          weth,
63          outputWeth,
64          uint64(block.timestamp)
65      );
66      vm.stopPrank();
67
68      int256 startingY = int256(weth.balanceOf(address(pool)));
69      int256 expectedDeltaY = int256(-1) * int256(outputWeth);
70
71      vm.startPrank(user);
72      pool.swapExactOutput(
73          poolToken,
74          weth,
75          outputWeth,
76          uint64(block.timestamp)
77      );
78      vm.stopPrank();
79
80      int256 endingY = int256(weth.balanceOf(address(pool)));
81      int256 actualDeltaY = int256(endingY) - int256(startingY);
82      assertEq(actualDeltaY, expectedDeltaY);
83  }
```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the x * y = k protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
1  -   swap_count++;
2  -   if (swap_count >= SWAP_COUNT_MAX) {
3  -       swap_count = 0;
4  -       outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000)
       ;
5  -   }
```

Medium

[M-1] `TSwapPool::deposit` function is missing deadline check, causing transactions to complete even after the deadline.

Description: `TSwapPool::deposit` function accepts a `deadline` parameter, which according to the documentation is "The deadline for the transaction to be completed by". However, within the function it is never used. As a consequence, operations that add liquidity to the pool might be executed at unexpected times, in market conditions where the deposit rate is unfavorable.

Impact: Transactions could be sent when market conditions are unfavorable to deposit even after adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Recommended Mitigation: Consider making the following change to the function:

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
    revertIfZero(wethToDeposit)
+   revertIfDeadlinePassed(deadline)
    returns (uint256 liquidityTokensToMint)
{}
```

Low

[L-1] `TSwapPool::LiquidityAdded` event has parameters out of order.

Description: When the `LiquidityAdded` is emitted in the `TSwapPool::_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the third parameter, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to off-chain functions potentially malfunctioning.

Recommended Mitigation:

```
- emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+ emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given.

Description: The `swapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `output` it is never assigned a value, nor uses an explicit return statement.

Impact: The return value will always be 0, giving incorrect information to the caller.

Proof of Concept:

PoC

```
function testSwapExactInputReturnsIncorrectResult() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 expected = 9e18;

    vm.startPrank(user);
    poolToken.approve(address(pool), 10e18);
    uint256 actualReturnValue = pool.swapExactInput(
        poolToken,
        10e18,
        weth,
        expected,
        uint64(block.timestamp)
    );
    vm.stopPrank();

    console2.log("Actual Return: ", actualReturnValue);
    assertApproxEqAbs(expected, actualReturnValue, 1);
    assertEq(actualReturnValue, 0);
}
```

Test result:

```
[FAIL: assertion failed: 900000000000000000 !~= 0 (max delta: 1, real
   delta: 900000000000000000)]
   testSwapExactInputReturnsIncorrectResult() (gas: 420353)
Logs:
  Actual Return:  0
```

Recommended Mitigation:

```
function swapExactInput(
    IERC20 inputToken,
```

```
3       uint256 inputAmount,
4       IERC20 outputToken,
5       uint256 minOutputAmount,
6       uint64 deadline
7   )
8       public
9       revertIfZero(inputAmount)
10      revertIfDeadlinePassed(deadline)
11      returns (
12          uint256 output
13      )
14  {
15      uint256 inputReserves = inputToken.balanceOf(address(this));
16      uint256 outputReserves = outputToken.balanceOf(address(this));
17
18  -   uint256 outputAmount = getOutputAmountBasedOnInput(
19  +   output = getOutputAmountBasedOnInput(
20          inputAmount,
21          inputReserves,
22          outputReserves
23      );
24
25  -   if (outputAmount < minOutputAmount) {
26  -       revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
27  -   }
28
29  +   if (output < minOutputAmount) {
30  +       revert TSwapPool__OutputTooLow(output, minOutputAmount);
31  +   }
32
33  -   _swap(inputToken, inputAmount, outputToken, outputAmount);
34  +   _swap(inputToken, inputAmount, outputToken, output);
35  }
```

Informational

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is used nowhere and should be removed.

```
1  - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking zero address check.

```
1  + error PoolFactory__InvalidWethAddress(address wethAddress);
2  constructor(address wethToken) {
3  +       if (wethToken == address(0)) {
```

```
+            revert PoolFactory__InvalidWethAddress(wethToken);
+        }
         i_wethToken = wethToken;
     }
```

```
+ error TSwapPool__InvalidTokenAddress();
constructor(
    address poolToken,
    address wethToken,
    string memory liquidityTokenName,
    string memory liquidityTokenSymbol
) ERC20(liquidityTokenName, liquidityTokenSymbol) {
+   if (poolToken == address(0) || wethToken == address(0)) {
+   revert TSwapPool__InvalidTokenAddress();
}
    i_wethToken = IERC20(wethToken);
    i_poolToken = IERC20(poolToken);
}
```

[I-3] `PoolFactory::createPool` should use `.symbol()` for `liquidityTokenSymbol` instead of `.name()`.

```
string memory liquidityTokenSymbol = string.concat(
    "ts",
-   IERC20(tokenAddress).name()
+   IERC20(tokenAddress).symbol()
);
```

[I-4] `PoolFactory::PoolCreated(address,address)` has address parameters but no `indexed` parameters.

Description: Indexing event parameters enables efficient off-chain filtering.

Impact: The `PoolCreated` event has address parameters but none are indexed. Off-chain tools cannot efficiently filter addresses of newly created pools and their respective addreses.

Recommended Mitigation: Add the `indexed` keyword to address parameters in events to enable efficient off-chain filtering.

```
+ event PoolCreated(address indexed tokenAddress, address indexed
    poolAddress);
```

[I-5] `TSwapPool::TSwapPool__WethDepositAmountTooLow` accepts `minimumWethDeposit` which is a constant variable and therefore not required to be emitted.

```
error TSwapPool__WethDepositAmountTooLow(
-   uint256 minimumWethDeposit,
    uint256 wethToDeposit
);
```

[I-6] The `poolTokenReserves` local variable in `TSwapPool::deposit` is used nowhere and should be removed.

```
- uint256 poolTokenReserves = i_poolToken.balanceOf(address(this));
```

[I-7] `TSwapPool::deposit` function updates the `liquidityTokensToMint` variable to return after making a potential external call, violating the CEI Pattern.

While it is not an updating of a state variable, it would be better to follow CEI in any case, which will increase the code maturity.

```
....
....
....
    } else {
        // This will be the "initial" funding of the protocol. We are starting from blank here!
        // We just have them send the tokens in, and we mint liquidity tokens based on the weth
+       liquidityTokensToMint = wethToDeposit;
        _addLiquidityMintAndTransfer(
            wethToDeposit,
            maximumPoolTokensToDeposit,
            wethToDeposit
        );
-       liquidityTokensToMint = wethToDeposit;
    }
```

[I-8] `TSwapPool::getOutputAmountBasedOnInput`, `TSwapPool::getInputAmountBasedOnOutput` and `TSwapPool::swapExactInput` are missing a documentation (NatSpec) while they are the core functions of the entire protocol.

Description: Lack of a documentation decreases readability of a codebase and is not a good practice in development.

Impact: Developers who will read this code later will spend significantly more time understanding the purpose of the functions.

Recommended Mitigation: Write the documentation (NatSpec) to those three functions.

[I-9] The variables to calculate fees within the `TSwapPool.sol` are not declared as constants.

```
+ uint256 constant FEE = 997;
+ uint256 constant FEE_PRECISION = 1000;

    function getOutputAmountBasedOnInput(
        uint256 inputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(inputAmount)
        revertIfZero(outputReserves)
        returns (uint256 outputAmount)
    {
+       uint256 inputAmountMinusFee = inputAmount * FEE;
        uint256 numerator = inputAmountMinusFee * outputReserves;
+       uint256 denominator = (inputReserves * FEE_PRECISION) + inputAmountMinusFee;
        return numerator / denominator;
    }

    function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
+           ((inputReserves * outputAmount) * FEE_PRECISION) /
+           ((outputReserves - outputAmount) * FEE);
    }
```

[I-10] `TSwapPool::swapExactOutput` function should be declaread as an `external` function, due to the fact that it is used nowhere within the contract.

Additionally, because of the incorrect implementation of the `TSwapPool::sellPoolTokens` function, `TSwapPool::swapExactInput` should be used instead.

```
    function swapExactOutput(
        IERC20 inputToken,
        IERC20 outputToken,
        uint256 outputAmount,
        uint64 deadline
    )
-       public
+       external
        revertIfZero(outputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 inputAmount)
    {}
```

[I-11] The `TSwapPool::swapExactOutput` NatSpec is missing the `deadline` parameter, causing the NatSpec to be incomplete.

```
/**
     * @notice figures out how much you need to input based on how much
     * output you want to receive.
     *
     * Example: You say "I want 10 output WETH, and my input is DAI"
     * The function will figure out how much DAI you need to input to
        get 10 WETH
     * And then execute the swap
     * @param inputToken ERC20 token to pull from caller
     * @param outputToken ERC20 token to send to caller
     * @param outputAmount The exact amount of tokens to send to caller
+    * @param deadline The deadline for the transaction to be completed
    by
     */
    function swapExactOutput(
        IERC20 inputToken,
        IERC20 outputToken,
        uint256 outputAmount,
        uint64 deadline
    )
        public
        revertIfZero(outputAmount)
        revertIfDeadlinePassed(deadline)
        returns (uint256 inputAmount)
    {
        uint256 inputReserves = inputToken.balanceOf(address(this));
```

```
25          uint256 outputReserves = outputToken.balanceOf(address(this));
26
27          inputAmount = getInputAmountBasedOnOutput(
28              outputAmount,
29              inputReserves,
30              outputReserves
31          );
32
33          _swap(inputToken, inputAmount, outputToken, outputAmount);
34      }
```

[I-12]: Large Numeric Literal

Large literal values multiples of 10000 can be replaced with scientific notation. Use `e` notation, for example: `1e18`, instead of its full numeric value.

1 Found Instance

- Found in src/TSwapPool.sol Line: 45

```
1       uint256 private constant MINIMUM_WETH_LIQUIDITY = 1
            _000_000_000;
```